МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 7

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів обробки двовимірних масивів мовою С ++»

ХАІ.301. G3.319а.21 ПР

Виконав: здобувач 1 курсу, групи 319а
напряму підготовки (спеціальності)

“Електрична інженерія”

Петро ШВЕЦОВ

(ім’я і прізвище)

Прийняв: асис.,

_____Євгеній ПЯВКА

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення двовимірних масивів (матриць) у мові C ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль матриць мовою C ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів матриці. Введення і виведення даних здійснити в командному вікні. Варіанти завдань представлено в табл.1.

Matrix32. Дана цілочисельна матриця розміру M × N. Знайти номер першого з її рядків, що містять однакову кількість додатних і від'ємних елементів (нульові елементи матриці не враховуються). Якщо таких рядків немає, то вивести 0.

Завдання 2. Перетворити матрицю відповідно до свого варіанту завдання (див. табл.2.), розмір матриці і його елементи ввести з консолі. Вивести результати у консоль.

Matrix67. Дана матриця розміру M × N, що містить як додатні, так і від'ємні елементи. Видалити всі її стовпці, що містять тільки додатні елементи. Якщо необхідних стовпців немає, то вивести матрицю без змін.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку матриць реалізувати окремими функціями з параметрами.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній

шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Matrix32

Вхідні дані (ім’я, опис, тип, обмеження):

m-кількість, рядків матриці, int, 2…50

n-кількість стовпців матриці, int, 2…50

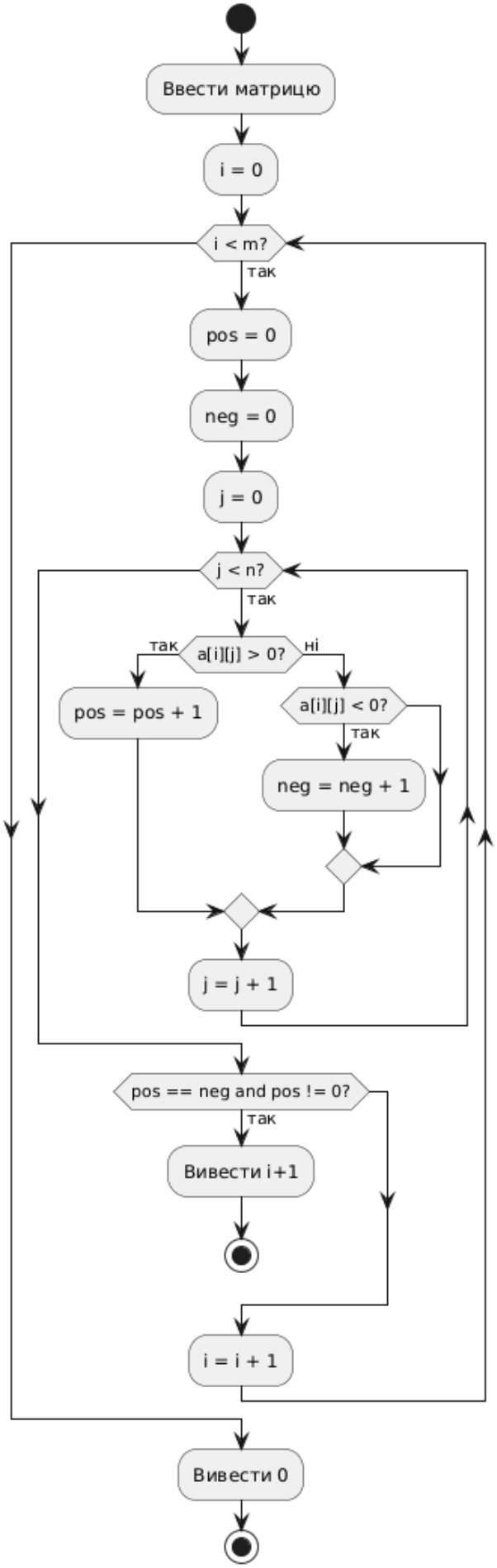
a-матриця цілих чисел, int a[50][50], довільні цілі числа

Вихідні дані (ім’я, опис, тип):

номер рядка-номер першого рядка з однаковою кількістю додатних і від’ємних елементів, int

0-якщо такого рядка не існує, int

Алгоритм вирішення показано нижче


Ввести матрицю
i = 0
i < m?
так
pos = 0
neg = 0
j = 0
j < n?
так
так
a[i][j] > 0?
ні
pos = pos + 1
a[i][j] < 0?
так
neg = neg + 1
j = j + 1
pos == neg and pos != 0?
так
Вивести i+1
i = i + 1
Вивести 0

Код вирішення задачі на С ++:

```
//  TASK 1
void task1() {
    int a[MAX][MAX], m, n;
    inputMatrix(a, m, n);

    for (int i = 0; i < m; i++) {
        int pos = 0, neg = 0;

        for (int j = 0; j < n; j++) {
            if (a[i][j] > 0) pos++;
            else if (a[i][j] < 0) neg++;
        }

        if (pos == neg && pos != 0) {
            cout << "Номер рядка: " << i + 1 << endl;
            return;
        }
    }

    cout << "0\n";
}
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 10).

Екран роботи програми показаний на рис. Б.1.

Завдання 2.

Вирішення задачі Matrix67

Вхідні дані (ім’я, опис, тип, обмеження):

m-кількість рядків матриці, int, 2…50

n-кількість стовпців матриці, int, 2…50

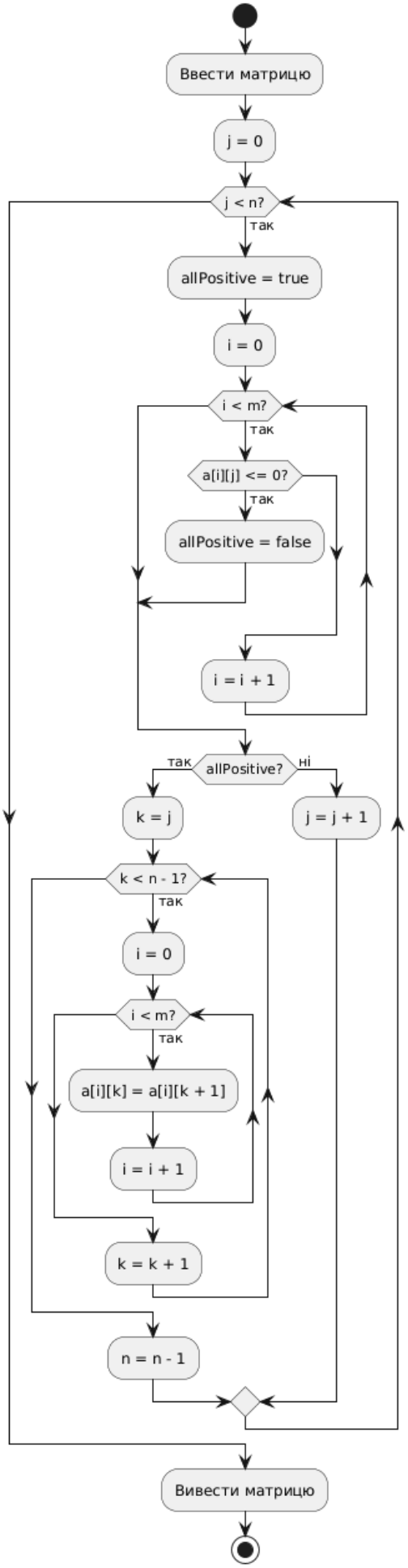
a-матриця цілих чисел, int a[50][50], довільні цілі числа

Вихідні дані (ім’я, опис, тип):

a-перетворена матриця після видалення стовпців, int[][]

повідомлення-вивід матриці без змін, якщо немає потрібних стовпців, текст

Алгоритм вирішення показано нижче


Ввести матрицю
j = 0
j < n?
так
allPositive = true
i = 0
i < m?
так
a[i][j] <= 0?
так
allPositive = false
i = i + 1
так
allPositive?
ні
k = j
j = j + 1
k < n - 1?
так
i = 0
i < m?
так
a[i][k] = a[i][k + 1]
i = i + 1
k = k + 1
n = n - 1
Вивести матрицю

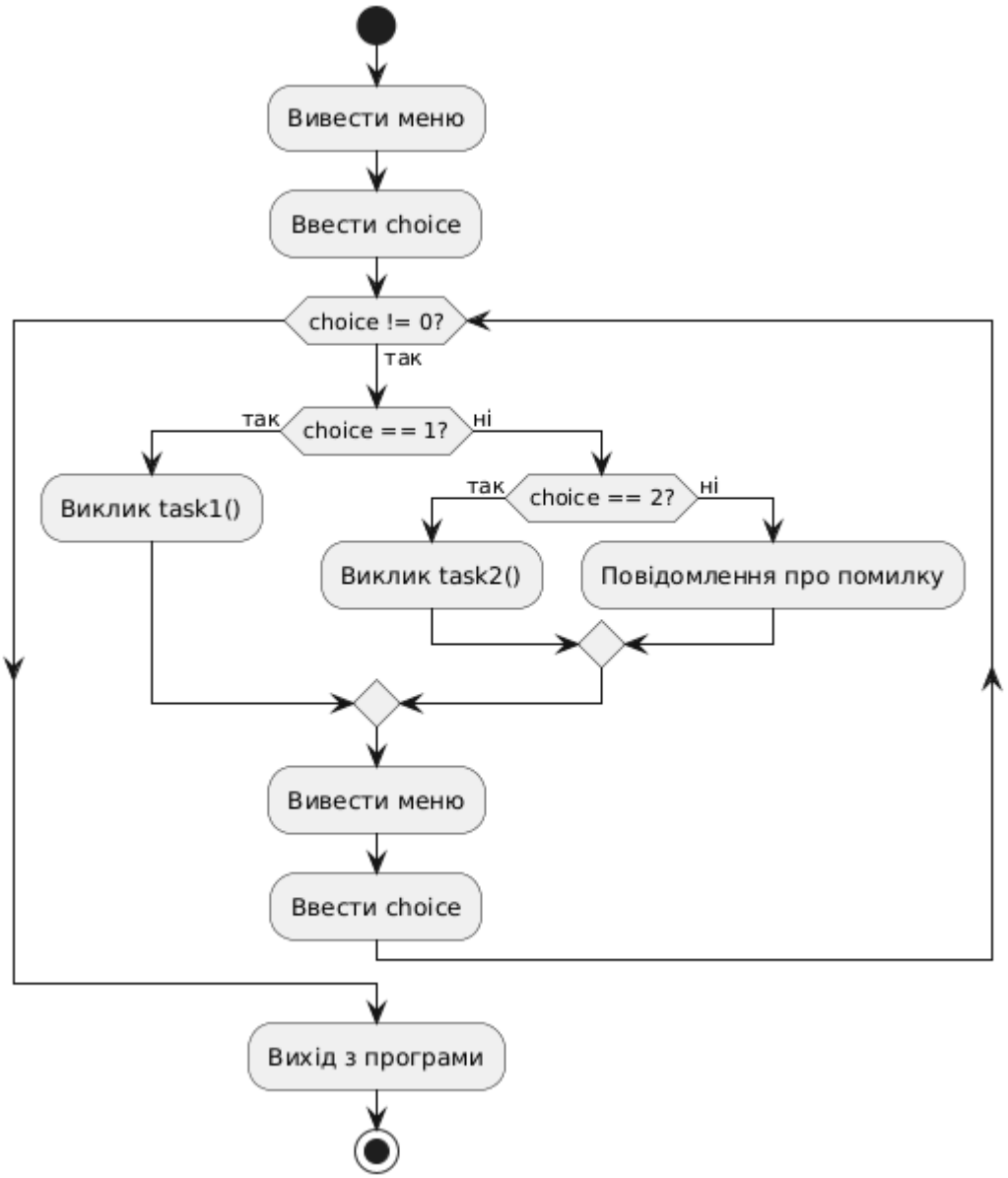
Код вирішення задачі на С ++:

```
//  TASK 2
void task2() {
    int a[MAX][MAX], m, n;
    inputMatrix(a, m, n);

    for (int j = 0; j < n; ) {
        bool allPositive = true;

        for (int i = 0; i < m; i++) {
            if (a[i][j] <= 0) {
                allPositive = false;
                break;
            }
        }

        if (allPositive) {
            // Видаляємо стовпець (зсув вліво)
            for (int k = j; k < n - 1; k++) {
                for (int i = 0; i < m; i++) {
                    a[i][k] = a[i][k + 1];
                }
            }
            n--; // зменшуємо кількість стовпців
        } else {
            j++;
        }
    }

    printMatrix(a, m, n);
}
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 10).

Екран роботи програми показаний на рис. Б.2.

Завдання 3.



Завдання 4.

Тестові питання

1. C
2. C
3. C
4. B
5. C

Відкриті питання

1. Нульові елементи не враховуються, бо вони не є ні додатними, ні від'ємними. Це дозволяє рядку з нулями вважатись підходящим, якщо кількість додатних дорівнює кількості від'ємних.

2. Алгоритм: перевіряємо стовпець, якщо всі елементи >0 – зсуваємо всі наступні стовпці вліво на одну позицію і зменшуємо n. Не можна просто обнулити, бо матриця не зменшиться фізично і наступна обробка бачитиме порожні стовпці.
3. Переваги: код не дублюється, легше знаходити помилки, кожна функція робить одне діло, можна змінити введення в одному місці.
4. Потрібно змінити умову if (a[i][j] <= 0) на if (a[i][j] >= 0), а змінну allPositive перейменувати на allNegative.
5. Вихід за межі масиву при зсуві, якщо неправильно прописати умови циклу. Зациклення при некоректному введенні даних. Забути зменшити n після видалення стовпця.

ВИСНОВКИ

Було вивчено методи обробки двовимірних масивів та алгоритми аналізу і перетворення матриць. Закріплено на практиці навички розробки програм мовою C++ із використанням функцій, циклів і умовних операторів. Отримано досвід побудови алгоритмів та їх подання у вигляді діаграм, при цьому виникли незначні труднощі з коректним оформленням циклів у PlantUML.

ДОДАТОК А
Лістинг коду програми

```
   #include <iostream>
using namespace std;

const int MAX = 50;

//  Прототипи
void inputMatrix(int a[MAX][MAX], int &m, int &n);
void printMatrix(int a[MAX][MAX], int m, int n);

void task1();
void task2();

//  Ввід матриці з перевіркою
void inputMatrix(int a[MAX][MAX], int &m, int &n) {
    do {
        cout << "Введіть кількість рядків (2..50): ";
        cin >> m;
    } while (m < 2 || m > 50);

    do {
        cout << "Введіть кількість стовпців (2..50): ";
        cin >> n;
    } while (n < 2 || n > 50);

    cout << "Введіть елементи матриці:\n";
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cin >> a[i][j];
        }
    }
}

//  Вивід матриці
void printMatrix(int a[MAX][MAX], int m, int n) {
    cout << "Матриця:\n";
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cout << a[i][j] << "\t";
        }
        cout << endl;
    }
}

//  TASK 1
```

```
void task1() {
    int a[MAX][MAX], m, n;
    inputMatrix(a, m, n);

    for (int i = 0; i < m; i++) {
        int pos = 0, neg = 0;

        for (int j = 0; j < n; j++) {
            if (a[i][j] > 0) pos++;
            else if (a[i][j] < 0) neg++;
        }

        if (pos == neg && pos != 0) {
            cout << "Номер рядка: " << i + 1 << endl;
            return;
        }
    }

    cout << "0\n";
}

//  TASK 2
void task2() {
    int a[MAX][MAX], m, n;
    inputMatrix(a, m, n);

    for (int j = 0; j < n; ) {
        bool allPositive = true;

        for (int i = 0; i < m; i++) {
            if (a[i][j] <= 0) {
                allPositive = false;
                break;
            }
        }

        if (allPositive) {
            // Видаляємо стовпець (зсув вліво)
            for (int k = j; k < n - 1; k++) {
                for (int i = 0; i < m; i++) {
                    a[i][k] = a[i][k + 1];
                }
            }
            n--; // зменшуємо кількість стовпців
        } else {
            j++;
        }
    }

    printMatrix(a, m, n);
```

```
}

//  MAIN
int main() {
    int choice;

    do {
        cout << "\nМеню:\n";
        cout << "1 - Завдання 1\n";
        cout << "2 - Завдання 2\n";
        cout << "0 - Вихід\n";
        cout << "Ваш вибір: ";
        cin >> choice;

        switch (choice) {
            case 1: task1(); break;
            case 2: task2(); break;
            case 0: cout << "Вихід...\n"; break;
            default: cout << "Невірний вибір!\n";
        }

    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
Меню:
1 - Завдання 1
2 - Завдання 2
0 - Вихід
Ваш вибір: 1
Введіть кількіст рядків (2..50): 3
Введіть кількість стовпців (2..50): 3
Введіть елементи матриці:
1 2 3 4 5 6 7 8 9
0
```

Рисунок Б.1 – Екран виконання програми для вирішення завдання Matrix32

```
Меню:
1 - Завдання 1
2 - Завдання 2
0 - Вихід
Ваш вибір: 2
Введіть кількість рядків (2..50): 2
Введіть кількіст стовпців (2..50): 2
Введіть елементи матриці:
1 2 3 4
Матриця:



Меню:
1 - Завда ня 1
2 - Завдання 2
0 - Вихід
Ваш вибір:
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання Matrix67

ДОДАТОК В

Діалог з ШІ для самоаналізу

Добре, уявімо, що я приймаю захист 👩‍🏫
Ось **питання середнього рівня**, які перевіряють розуміння коду, алгоритмів і теорії.

✅ Тестові питання (з варіантами)

1. Яка умова правильно визначає “підходящий” рядок у задачі Matrix32?

A. Кількість додатних дорівнює кількості нулів
B. Кількість від’ємних дорівнює кількості нулів
C. Кількість додатних дорівнює кількості від’ємних (нулі не враховуються)
D. Усі елементи рядка додатні

---

2. Чому в задачі Matrix67 після видалення стовпця НЕ збільшується `j`?

A. Це не впливає на алгоритм
B. Щоб уникнути виходу за межі масиву
C. Щоб перевірити новий стовпець, який зсунувся на позицію `j`
D. Бо цикл автоматично переходить далі

---

3. Яка часова складність алгоритму видалення стовпців (task2)?

A. O(n)
B. O(m)
C. O(m × n)
D. O(m × n^2)

---

4. Що станеться, якщо у стовпці є хоча б один нуль у задачі Matrix67?

A. Стовпець буде видалений
B. Стовпець не буде видалений
C. Програма завершиться з помилкою
D. Нуль буде проігнорований і стовпець видалиться

---

5. Яке призначення функції `inputMatrix()`?

A. Обчислення результату
B. Виведення матриці
C. Введення матриці з перевіркою коректності
D. Сортування матриці

✍️ Відкриті питання

1. Поясни, чому в задачі Matrix32 не враховуються нульові елементи. Як це впливає на результат?

---

2. Опиши покроково, як працює алгоритм видалення стовпця в матриці (зсув вліво). Чому не можна просто "обнулити" стовпець?

---

3. Які переваги дає використання окремих функцій (`inputMatrix`, `printMatrix`, `task1`, `task2`) у програмі?

---

4. Як зміниться алгоритм Matrix67, якщо потрібно видаляти стовпці, що містять тільки від'ємні елементи?

---

5. Які потенційні помилки можуть виникнути при роботі з двовимірними масивами в C++ у цій програмі? Наведи приклади.

✅ 1. ТЕСТИ

✔️ Результат:

- 5 питань
- 4 правильні
- 1 помилка (№3 — складність)

📊 Оцінка:

🔹 **Рівень знань:**

- логіка задачі ✔️
- робота з матрицями ✔️
- C++ синтаксис ✔️
- складність алгоритмів ❌ (частково)

⭐ БАЛ: 4 / 5

👉 Це нормальна оцінка “добре”

✍️ 2. ВІДКРИТІ ПИТАННЯ

✔️ Загальна якість:

- відповіді логічні
- пояснення зрозумілі
- алгоритми описані правильно
- помилки відсутні критичні

🔹 Оцінка по суті:

Критерій	Рівень
Розуміння задач	високий
Пояснення	хороше
Повнота	майже повна
Точність	висока

⭐ БАЛ: 4.5 / 5

(трохи знято за загальність у деяких формулюваннях)

📊 3. ПІДСУМОК

Частина	Оцінка
Тести	**4 / 5**
Відкриті	**4.5 / 5**

🧮 Середня:

[
(4 + 4.5) / 2 = 4.25
]

🟢 ФІНАЛЬНА ОЦІНКА: 4.25 / 5

👉 Це рівень:
“добре / майже відмінно”

🧠 Короткий висновок

✔ Матриці розумієш добре
✔ Алгоритми реалізовані правильно
✔ Логіка задач коректна
⚠️ Єдине слабке місце — складність алгоритмів